

Una API de productos, verbo por verbo

Laboratorio en vivo. El profesor arma desde cero, en IntelliJ, un modelo `Producto`, un controlador y una lista en memoria que hace de base de datos falsa. Va probando cada endpoint en Insomnia: primero el que lista, luego el que busca por id, después el que crea y al final el que borra. En el camino se topa con un error de `DataSource` y lo resuelve enfrente del grupo.

Cada endpoint lleva el color de su verbo: `GET` `POST` `PUT` `DELETE`. El `PUT` es el que no llegó a escribir. Los botones con `?` explican los términos que dio por sabidos, y al final hay un cliente de práctica para repetir las pruebas de la clase. Hay seis ejercicios.

Lo que salió de esta clase

01:31 a 58:50

Lo construido: el modelo `Producto` y el `ProductoController` con cuatro endpoints: `GET` lista, `GET` por id, `POST` y `DELETE`.

≈1:04:00

Falta el `PUT`. Dijo "ya es la misma lógica". No lo dejó como tarea, pero es la que cierra el CRUD.

arrastre

Convertor de temperaturas (clase 8): en esta transcripción no aparece que lo hayan revisado.

cierre

"Nos vemos el próximo miércoles, y nos vemos en laboratorio." Ver la nota de abajo antes de fiarte de esa frase.



Sobre los minutos y las citas. Los minutos vienen por bloques, no por línea. Las frases entre comillas siguen lo que dijo el profesor: fíate del sentido, no de que sean palabra por palabra.



Fecha y cosas que no cuadran. Dice "la propiedad que le pusimos ayer era `server.port`", y la clase 8 (9 de septiembre) terminó con laboratorio al día siguiente, así que esta cae el **jueves 10**. Confírmalo. La frase de cierre ("el próximo miércoles, y nos vemos en laboratorio") es idéntica a la de la clase 7 y aquí ya estamos en laboratorio; puede ser un calco. También dice "ya vamos tres de cinco métodos" cuando lleva dos.

BLOQUE 1 · MINUTO 00:00 A 01:30

Retomar el proyecto y cambiar el puerto

Abre la documentación de Spring Boot, cierra el `HELP.md` que ya no necesita y va al archivo de configuración.

Muestra en el navegador la guía de inicio, con Spring Web, Spring Data JPA, DevTools y Validation, y de ahí sigue en `application.properties`. La propiedad de ayer era `server.port`. Por defecto es 8080; ahora define 8083, "por poner un número".

```
server.port=8083
```

! **Dependencias que esta clase da por puestas.** Hoy se usan Spring Data JPA, el conector de MySQL y Lombok. En la clase 8 solo se marcaron Web y DevTools, y al profesor se le olvidó Lombok. Aquí dice que "en las dependencias de ayer metimos Lombok". No pude reconciliar las dos versiones. Si tu proyecto no las tiene, agrégalas al `pom.xml` antes de seguir.

BLOQUE 2 · MINUTO 01:31 A 03:30

El modelo: la clase `Producto`

Anticipa un error, "un errorzote", y decide dejarlo pasar para corregirlo después. Mientras tanto, crea el paquete y la clase del modelo.

"Casi es un hecho de que les va a marcar un errorzote. Entonces, hay que agregar por lo menos otras cuatro propiedades, pero vamos a dejarlo que nos genere el error y ya vemos cómo lo corregimos."

Nuevo paquete `modelo`, dentro una clase `Producto`. Empieza con `implements Serializable` y tres atributos:

```
public class Producto implements Serializable {  
    private Long id;  
    private String nombre;  
    private double precio;  
}
```

"Lo demás que ustedes quieran agregarle."

BLOQUE 3 · MINUTO 03:31 A 08:15

Todo lo repetitivo, a mano con el IDE

Antes de usar Lombok, hace la clase completa con el menú **Código** → **Generar**. Es la versión larga de lo que luego se resuelve con tres anotaciones.

1. **Constructor sin argumentos:** Código → Generar → constructor, y "Select None" para no marcar ningún atributo.
2. **Constructor con todos los atributos:** mismo menú, los selecciona todos.
3. **Getters y setters:** por separado, o de una vez con "Getters and Setters".
4. `equals()` y `hashCode()` : "a final de cuentas sabemos que el ID es único, que trabaje contra el ID".
5. `toString()` : "no lo quiero concatenado, quiero que trabaje con un `StringBuilder`".

— ¿Por qué `equals()` y `hashCode()` solo con el id?

Dos productos se consideran "el mismo" si tienen el mismo id, aunque el nombre o el precio hayan cambiado. Es lo que se espera de una entidad con identificador único. Los dos métodos deben ir juntos: si dos objetos son iguales según `equals`, tienen que dar el mismo `hashCode`.

Cuando termina dice "ya tenemos la clase completita" y la reformatea, "porque no me gusta cómo aparece". Espera a que el grupo se ponga al día: "ustedes me dicen cuando ya hayan llegado hasta ahí".

DIGRESIÓN · MINUTO 08:16 A 12:45

Dónde se buscan las dependencias

Mientras el grupo copia el código, aprovecha para mostrar cómo se agrega una librería a Maven.

Abre en el navegador el **repositorio de Maven**, busca una librería cualquiera (usó **JFreeChart** como ejemplo) y muestra que ahí mismo te dan el fragmento para pegar en el proyecto. Lo cierra con "esto ya es otro tema".

— ¿Cómo se agrega una dependencia a un proyecto Maven?

Buscas la librería en el repositorio, eliges una versión y copias el bloque `<dependency>` que te muestra. Lo pegas dentro de `<dependencies>` en el `pom.xml` y recargas Maven; el IDE descarga la librería. Es lo mismo que hace Spring Initializr al marcar casillas, pero a mano.

BLOQUE 4 · MINUTO 12:46 A 15:00

Lombok reemplaza todo lo anterior

Borra de golpe lo que acaba de generar y lo sustituye por tres anotaciones.

*"Vamos a agarrar todo esto, de aquí hasta acá, y se va. Vamos a borrarlo. Y simplemente vamos a ponerle un **@Data**. [...] Nos hace falta un constructor: **@NoArgsConstructor**. [...] Y vamos con el **@AllArgsConstructor**."*

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Producto implements Serializable {
    private Long id;
    private String nombre;
    private double precio;
}
```

Un alumno preguntó si ese paquete no lo tenían de ayer. Respuesta: sí, "en las dependencias de ayer metimos Lombok, para evitarnos conflictos y hacer el código más rápido".

- i **Un detalle mío, no del profesor.** Hizo `equals` y `hashCode` a mano solo con el id. `@Data` los regenera con **todos** los campos, así que el criterio cambia sin que nadie lo mencione. Aquí no afecta, porque el controlador compara ids directamente. Si quisieras conservar la igualdad por id con Lombok, se hace con `@EqualsAndHashCode(onlyExplicitlyIncluded = true)` y `@EqualsAndHashCode.Include` sobre el campo `id`.

El `Producto` a mano y con Lombok

Cuatro preguntas sobre lo que pidió generar y cómo quedó al final.
Contesta las cuatro y dale a **Revisar**.

¿Cómo se llama el paquete del modelo y la clase que crea?

modelo y Producto

dto y Producto

entity y Item

¿Qué interfaz implementa la clase?

Serializable

Comparable

Cloneable

¿Contra qué atributo trabajan `equals()` y `hashCode()` cuando los genera a mano?

nombre

id

precio

¿Cómo pide que se genere el `toString()`?

Concatenado con +

Con un `StringBuilder`

No lo genera

Revisar

Reiniciar

EJERCICIO 2 · LOMBOK

Qué cambia con las tres anotaciones

Tres preguntas sobre lo que se borró y lo que se puso.

¿Qué tres anotaciones ponen en lugar de constructores, getters, setters y métodos?

@Data, @NoArgsConstructor y @AllArgsConstructor

@Getter, @Setter y @ToString

@Builder y @Value

¿Qué genera @AllArgsConstructor ?

El constructor sin argumentos

El constructor con todos los elementos

Los getters

¿Para qué dice que usa Lombok?

Para evitar conflictos y hacer el código más rápido

Porque Spring lo exige

Para poder usar records

Revisar

Reiniciar

BLOQUE 5 · MINUTO 15:01 A 17:30

El controlador

Vuelve al paquete base, crea un paquete nuevo para los controladores y empieza con las anotaciones de la clase 8.

Paquete `controlador`, clase `ProductoController`. Las anotaciones son "igual a las de ayer": `@RestController`, y un `@RequestMapping` con la ruta base. Para "mantener la costumbre", la ruta lleva versión:

```
@RestController
@RequestMapping("/api/v1/productos")
public class ProductoController {

}
```

— ¿Por qué `/api/v1/` en la ruta?

El `v1` es la **versión** de la API. Si un día cambias cómo responde algún endpoint y no quieres romper a quien ya lo usa, publicas `/api/v2/...` y las dos conviven. En la clase

7 lo mencionó como parte de lo que sube el nivel de madurez de una API.

BLOQUE 6 • MINUTO 17:31 A 20:50

Una lista en memoria como base de datos falsa

No hay base de datos todavía. Los datos son una lista dentro del propio controlador, con cinco productos de ejemplo.

```
private final List<Producto> productos = new ArrayList<> (List.of(
    new Producto(1L, "Laptop 1", 5000),
    new Producto(2L, "Laptop 2", 6000),
    new Producto(3L, "Laptop 3", 7000)
    // ...hasta 5 productos
));
```

Copia y pega hasta tener cinco. Luego el IDE marca un error y el profesor le pregunta al grupo cómo corregirlo. Un alumno responde: importando la lista y `Producto`. Hace falta eso, pero no es lo que rompía:

*"Hay que castigarlo... porque es un valor entero y el tipo es **Long**. Entonces le decimos que es un **Long** poniéndole una **L** al final."*

— ¿Por qué `1L` y no `1` ?

El literal `1` es un `int`, y el constructor pide un `Long`. Java no convierte solo un `int` a `Long` (el objeto envoltorio). Con `1L` el literal ya es `long` y se empaqueta a `Long`. Con `5000` en el precio no hay problema, porque el campo es `double` y un entero sí se ensancha a `double`.

— ¿Por qué envolver `List.of(...)` en un `new ArrayList<>(...)` ?

`List.of` devuelve una lista **inmutable**: no deja agregar ni quitar. Como el `POST` va a hacer `add` y el `DELETE` va a hacer `removeIf`, la lista tiene que ser modificable. `new ArrayList<>(...)` copia los elementos a una lista que sí lo es. El profesor no lo explicó en la clase; lo puso así desde el inicio.

GET /api/v1/productos

BLOQUE 7 • MINUTO 20:51 A 25:25

Primer endpoint: listar productos

El más simple. No recibe nada y devuelve toda la lista.

```
@GetMapping
public ResponseEntity<List<Producto>> listarProductos() {
    return ResponseEntity.ok(productos);
}
```

"En teoría no tenemos más nada que hacer." Le da **Run** y anticipa que va a salir "el kilo de errores".

EJERCICIO 3 · ARMA EL CONTROLADOR

ProductoController, en orden

Toca las líneas en el orden en que las escribió, de arriba abajo. Si te equivocas te doy una pista; toca una línea ya puesta para quitarla.

Aquí se va armando el controlador...

```
} @RestController @GetMapping public class ProductoController {
@RequestMapping("/api/v1/productos")
private final List<Producto> productos = new ArrayList<>(List.of(new Producto(1L,
"Laptop 1", 5000), ...));
} return ResponseEntity.ok(productos);
public ResponseEntity<List<Producto>> listarProductos() {
```

Reiniciar

BLOQUE 8 · MINUTO 25:26 A 32:00

El error del DataSource

La aplicación no arranca. La consola muestra una excepción y él la usa para explicar por qué.

"Y si les aparece algo como eso... vamos bien. ¿Por qué? Bueno, nos dice que trató de arrancar, no pudo, y nos da una referencia: 'Oye, mira, no has configurado un DataSource! [...] Pero no estamos usando base de datos. La respuesta es no, pero por el simple hecho de haber agregado la dependencia de Spring Data, hay que configurarlo."

La solución es agregar cuatro propiedades al `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/bd
spring.datasource.username=root
spring.datasource.password=<la contraseña de tu MySQL>
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Dictó una contraseña de ejemplo; aquí la dejo como marcador. Vuelve a correr y esta vez Tomcat arranca en el **8083** sin errores.

– ¿Qué es un `DataSource` y por qué lo exige Spring Data?

Es el objeto que sabe conectarse a la base de datos: la URL, el usuario, la contraseña y el driver. Al agregar **Spring Data JPA**, Spring asume que vas a usar una base y trata de crear ese objeto al arrancar. Si no le dices dónde está, se detiene. Por eso da error aunque el código de hoy nunca toque una base.

Cada propiedad: `url` es la dirección de la base (servidor, puerto 3306 y nombre `bd`); `username` y `password` son las credenciales; `driver-class-name` es la clase Java del conector de MySQL.

"En teoría ya tenemos conectado Spring con la base de datos... aunque la base no exista todavía. Si les corrió, literal, es porque algo hicieron mal."

- i **Dos notas más sobre esto.** La última frase parece hablar de quien arrancó sin error antes de configurar el `DataSource`: sin la dependencia de Data, no habría error. Y sobre "aunque la base no exista": con JPA y MySQL, Spring suele intentar conectarse al arrancar. Si a ti te truena porque no hay un MySQL corriendo o la base `bd` no existe, no es un error tuyo; lo más probable es que él sí tuviera uno activo. Otra salida, que él no mencionó, es quitar la dependencia de Data mientras no la uses.

EJERCICIO 4 · EL ARRANQUE

Puerto, DataSource y propiedades

Cuatro preguntas sobre por qué falló el primer arranque y cómo se arregló.

¿Por qué no arrancó la aplicación la primera vez?

Falta la anotación @GetMapping

No se había configurado un DataSource

El puerto 8083 estaba ocupado

El proyecto no usa base de datos. ¿Por qué entonces pide el DataSource?

Porque Tomcat siempre lo pide

Por haber agregado la dependencia de Spring Data

Porque el modelo implementa Serializable

¿Cuáles son las cuatro propiedades que agrega?

url, username, password y driver-class-name

port, name, host y path

url, table, schema y dialect

Con la configuración lista, ¿en qué puerto arranca Tomcat?

8080

8082

8083

Revisar

Reiniciar

GET /api/v1/productos

BLOQUE 9 · MINUTO 32:01 A 36:50

Probar el listado en Insomnia

Primera prueba real del endpoint, con una herramienta gráfica.

Abre **Insomnia**, "insisto, porque me gustó". Ustedes usen Postman, Thunder Client "o el que quieran". Pasos:

1. Crear una colección, **Productos**.
2. Nuevo request, **Mostrar Lista Productos**.
3. URL: `http://localhost:8083/api/v1/productos`. Verbo **GET**.
4. Darle **Send**.

La respuesta es un 200 OK con el arreglo JSON de las cinco laptops, la misma lista que se creó en la clase. "Ya nos está devolviendo la lista de productos."

- i **Esto resuelve una duda de la clase 8.** Allí el profesor usaba una herramienta gráfica sin nombrarla, y no pude saber si era Postman o Insomnia. Aquí dice que prefiere Insomnia. Es probable que fuera la misma, pero la clase 8 no lo confirma.

GET /api/v1/productos/{id}

BLOQUE 10 · MINUTO 36:51 A 45:20

Segundo endpoint: buscar por id

Empieza por probar la ruta antes de programarla, para ver el 404, y luego escribe el método.

Duplica el request anterior, lo llama **Mostrar un producto** y agrega `/2` a la URL. La respuesta es un 404 Not Found. "Tiene sentido porque no lo hemos programado."

Regresa al código y escribe el método:

```
@GetMapping("/{id}")
public ResponseEntity<Producto> mostrarProducto(@PathVariable Long id) {
    Optional<Producto> producto = productos.stream()
        .filter(p -> p.getId().equals(id))
        .findFirst();
    if (producto.isPresent()) {
        return ResponseEntity.ok(producto.get());
    } else {
        return ResponseEntity.notFound().build();
    }
}
```

Guarda, reinicia el servidor y prueba: el id 2 devuelve 200 OK con la Laptop 2. El id 30, que no existe, devuelve 404 Not Found. "Funciona."

– ¿Qué es `@PathVariable` y en qué se distingue de `@RequestParam` ?

`@PathVariable` toma un valor que va **dentro de la ruta**: en `/productos/2`, el 2 es el `{id}`. `@RequestParam`, de la clase 8, toma un valor que va **después del signo de interrogación**: `?valor=10`. Por eso aquí el `@GetMapping` lleva `"/{id}"` entre llaves y allá no.

– ¿Qué hace ese `stream().filter(...).findFirst()` y qué es un `Optional` ?

Recorre la lista, se queda solo con los productos cuyo id coincide con el pedido, y toma el primero. Como puede que no haya ninguno, el resultado no es un `Producto` sino un `Optional<Producto>`: una caja que puede tener el producto o estar vacía. Con `isPresent()` se pregunta si la caja trae algo, y con `get()` se saca.

"Si no, devuélveme un `ResponseEntity.notFound().build()`."

POST /api/v1/productos

BLOQUE 11 · MINUTO 45:21 A 58:50

Tercer endpoint: crear un producto

El bloque más largo. Escribe el método y prueba el envío con un cuerpo JSON.

Dice "ya vamos tres de cinco métodos. Hace falta el PUT y el POST". Escribe primero el POST, sin cambiar la ruta:

```
@PostMapping
public ResponseEntity<Producto> crearProducto(@RequestBody Producto producto) {
    productos.add(producto);
    return ResponseEntity
        .created(URI.create("/api/v1/productos/" + producto.getId()))
        .body(producto);
}
```

El producto no se guarda en ninguna base: solo se agrega a la lista en memoria. Devuelve un **201 Created**, con la ruta del nuevo recurso y el objeto en el cuerpo.

— ¿Qué hace `@RequestBody` ?

Toma el **cuerpo** de la petición, que llega como JSON, y lo convierte en un objeto `Producto` haciendo coincidir los nombres de los campos (`id`, `nombre`, `precio`). Es lo que usan `POST` y `PUT` para recibir datos, porque un `GET` no lleva cuerpo.

En Insomnia clona la petición, cambia el verbo a **POST**, elige el tipo JSON y escribe el cuerpo "de lo que modelé":

```
{
  "id": 6,
  "nombre": "Laptop 6",
  "precio": 15000
}
```

Al enviar responde **201 Created**. Vuelve a "Mostrar lista" y el producto 6 ya está. "Mientras la aplicación no se detenga, ahí va a estar."

- i **Es el "happy path" de la clase 7, otra vez.** Este POST no valida nada: si mandas otro producto con id 6, lo agrega de nuevo y responde 201. Tampoco hay manejo de errores, que es justo lo que el profesor pidió controlar a mano. Aquí no lo hace porque es un ejemplo; en tu proyecto sí tendrías que.

DELETE /api/v1/productos/{id}

BLOQUE 12 · MINUTO 58:51 A 1:05:00

Cuarto endpoint: eliminar, y cierre

El último método de la clase y el cierre. El PUT queda en el aire.

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminarProducto(@PathVariable Long id) {
    productos.removeIf(p -> p.getId().equals(id));
    return ResponseEntity.noContent().build();
}
```

"No quiero que me devuelva nada", así que responde **204 No Content**. Por eso el tipo de retorno es `ResponseEntity<Void>` : no hay cuerpo.

En Insomnia crea una petición **DELETE** hacia `/api/v1/productos/4` .

Responde **204 No Content**. Al listar de nuevo, la lista "salta del 3 al 5": el 4 ya no existe.

"A final de cuentas estamos jugando con los métodos. Faltaría el PUT para actualizar, pero ya es la misma lógica. Sería todo por el día de hoy, no les doy más lata."

- i **Otra nota mía.** Este **DELETE** devuelve 204 aunque el id no exista, porque `removeIf` no falla cuando no encuentra nada. Es la misma limitación de "happy path" de arriba.

PENDIENTE SUGERIDO ≈1:04:00

El **PUT** para actualizar

No lo dejó como tarea. Lo dijo como lo que falta y como "la misma lógica" que el resto. Con esto el CRUD queda completo. Es lo natural para practicar antes de la siguiente clase.

"Faltaría el PUT para actualizar, pero ya es la misma lógica."

EJERCICIO 5 · CÓDIGOS DE RESPUESTA

Cada caso, con su código y su llamada

Cuatro casos de los que probó en Insomnia. Elige el código y la forma de devolverlo.

GET /api/v1/productos/30 , que no existe.

200, ResponseEntity.ok(null)

404, ResponseEntity.notFound().build()

204, ResponseEntity.noContent().build()

POST /api/v1/productos con un producto nuevo.

200, ResponseEntity.ok(...)

201, ResponseEntity.created(URI...).body(producto)

204, ResponseEntity.noContent().build()

DELETE /api/v1/productos/4 .

200, con el producto borrado

204, ResponseEntity.noContent().build()

404 siempre

Probó GET /productos/2 antes de programarlo y devolvió 404. ¿Por qué?

Porque el producto 2 no existe

Porque todavía no había ningún método mapeado a esa ruta

Porque falta el DataSource

Revisar

Reiniciar

LABORATORIO

Repite las pruebas de la clase

Un cliente de práctica que se comporta como el servidor de la clase. Es un simulador que corre en tu navegador, no un Spring de verdad. Sirve para repetir las cuatro pruebas y ver qué pasa si mandas algo que él no probó.

Completa las cinco pruebas

Elige el verbo, escribe la ruta (el servidor está en `localhost:8083`) y dale a **Send**. Las pruebas de abajo se marcan solas. **Reiniciar servidor** deja la lista como al arrancar, igual que reiniciar el proyecto.

GET

POST

PUT

DELETE

`http://localhost:8083/api/v1/producto`

Send

La respuesta aparece aquí.

Reiniciar servidor

- ☐ GET a la lista: responde 200 con las cinco laptops.
- ☐ GET al id 30, que no existe: responde 404.
- ☐ POST del producto 6: responde 201.
- ☐ DELETE al id 4: responde 204.
- ☐ GET a la lista otra vez: la lista salta del 3 al 5.

Los precios de las laptops 4 y 5 no se leen en la transcripción; puse 8000 y 9000 siguiendo el patrón. Los códigos 400, 405 y 500 que puede devolver este simulador no los probó él: salen de cómo se comporta Spring con ese código.

CÓDIGO DE LA CLASE

El proyecto de corrido

Los tres archivos como quedaron al final de la clase: el modelo con Lombok, el controlador con los cuatro endpoints y la configuración.

`application.properties`

```
server.port=8083

spring.datasource.url=jdbc:mysql://localhost:3306/bd
spring.datasource.username=root
spring.datasource.password=<la contraseña de tu MySQL>
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

`modelo/Producto.java`

```
package com.ipn.mx.modelo;

import java.io.Serializable;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Producto implements Serializable {
    private Long id;
    private String nombre;
    private double precio;
}
```

controlador/ProductoController.java

```

package com.ipn.mx.controlador;

import com.ipn.mx.modelo.Producto;
import java.net.URI;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/productos")
public class ProductoController {

    private final List<Producto> productos = new ArrayList<> (List.of(
        new Producto(1L, "Laptop 1", 5000),
        new Producto(2L, "Laptop 2", 6000),
        new Producto(3L, "Laptop 3", 7000),
        new Producto(4L, "Laptop 4", 8000),
        new Producto(5L, "Laptop 5", 9000)
    ));

    @GetMapping
    public ResponseEntity<List<Producto>> listarProductos() {
        return ResponseEntity.ok(productos);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Producto> mostrarProducto(@PathVariable Long id) {
        Optional<Producto> producto = productos.stream()
            .filter(p -> p.getId().equals(id))
            .findFirst();
        if (producto.isPresent()) {
            return ResponseEntity.ok(producto.get());
        } else {
            return ResponseEntity.notFound().build();
        }
    }

    @PostMapping
    public ResponseEntity<Producto> crearProducto(@RequestBody Producto producto) {
        productos.add(producto);
        return ResponseEntity
            .created(URI.create("/api/v1/productos/" + producto.getId()))
            .body(producto);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminarProducto(@PathVariable Long id) {
        productos.removeIf(p -> p.getId().equals(id));
    }
}

```

```
        return ResponseEntity.noContent().build();
    }

    // Falta: @PutMapping("/{id}") para actualizar
}
```

— ¿En qué se diferencia esta versión de lo que quedó en el pizarrón?

Son cabos sueltos del ritmo de la clase, no correcciones a lo que dijo:

- **Imports.** Los agregué para que compile. El IDE los pone solo, pero la transcripción no los muestra.
- **Paquete base.** Supuse `com.ipn.mx`, el de la clase 8. La transcripción solo dice "el paquete base".
- **Precios 4 y 5.** No se leen; puse 8000 y 9000 por el patrón.
- **Contraseña.** La dictó de ejemplo. Aquí va un marcador.
- `equals` y `hashCode`. A mano los hizo solo por id. Con `@Data` se regeneran con todos los campos, como dije en el bloque 4.
- **El comentario de `PUT`.** Es mío, para marcar lo que falta.

CIERRE

Checklist de la clase 9

Se guarda en tu navegador. Nada se sube a ningún lado ni se comparte con nadie más.

0 / 4

☐ Pendiente sugerido · escribir el PUT para actualizar

No lo dejó como tarea. Dijo que faltaba y que "ya es la misma lógica". Con él el CRUD queda completo: GET lista, GET por id, POST, PUT y DELETE.

☐ Reproducir el proyecto en tu máquina

Modelo Producto con Lombok, ProductoController con cuatro endpoints, application.properties con el puerto 8083 y las cuatro propiedades del DataSource. Prueba cada uno en Insomnia.

☐ Arrastre · terminar el conversor de temperaturas (clase 8)

Faltaban F a C, F a K, K a C y K a F. En esta transcripción no aparece que lo hayan revisado en el laboratorio.



Por confirmar · ¿hay clase o laboratorio el próximo miércoles?

Cerró con "nos vemos el próximo miércoles, y nos vemos en laboratorio", frase idéntica a la de la clase 7. Confírmalo con tu horario.